

Cursor Prompt — Bangle Retail MVC Website

Updated to include configurable adult and baby/kids bangle sizes, modern retail UI/UX, WhatsApp ordering, JazzCash and bank-card payment integration points.

You are an expert ASP.NET Core MVC developer and modern UI/UX designer.

Build a complete, production-style retail e-commerce website for selling bangles.

TECH STACK

- ASP.NET Core MVC
- Razor Views (.cshtml)
- C#
- Entity Framework Core
- SQL Server
- Bootstrap 5 or modern CSS
- JavaScript for interactive UI/UX
- Responsive design
- Use clean architecture and clean coding practices where practical

IMPORTANT:

This must be an MVC application using Controllers + Models + Razor Views.
Do NOT build this as a React, Angular, Vue, or Blazor application.

WEBSITE PURPOSE

The website is an online retail store specializing in bangles.

Customers should be able to:

- Browse bangles
- View individual products
- See product images
- See available sizes, including baby/children's sizes
- See price
- Select size
- Select quantity
- Add products to cart
- View cart
- Proceed to checkout
- Choose a payment method
- Contact the seller through WhatsApp
- Place an order

DESIGN / UI / UX

Create a premium, modern, elegant jewelry-store design.

The overall visual style should feel:

- Elegant
- Modern
- Premium
- Minimal
- Female-fashion oriented
- Clean
- Trustworthy

Use:

- Soft neutral background
- Elegant typography
- Subtle gold/rose-gold accents
- Rounded cards where appropriate
- High-quality product imagery
- Soft shadows
- Smooth hover animations
- Smooth transitions
- Micro-interactions
- Modern buttons
- Responsive navigation
- Sticky navigation where appropriate

Do not make the website look like a generic Bootstrap template.

HOME PAGE

Create a visually impressive homepage.

Sections:

1. Hero Section

- Large jewelry/bangle image
- Strong heading such as:
"Elegance on Every Wrist"
- Short description
- "Shop Now" CTA
- "Order on WhatsApp" CTA
- Modern entrance animation

2. Featured Products

- Display popular/new bangles
- Product cards
- Product image
- Product name
- Price
- Available sizes
- Quick View button
- Add to Cart button

3. Categories

Examples:

- Gold Bangles
- Bridal Bangles
- Traditional Bangles
- Stone Bangles
- Daily Wear
- Premium Collection
- Baby & Kids Bangles

4. Why Choose Us

Examples:

- Premium Quality
- Multiple Sizes
- Secure Payment
- Easy Ordering
- WhatsApp Support

5. Promotional section

Example:

"Find Your Perfect Pair"

6. Customer testimonials

7. Instagram/social-style gallery if appropriate

8. Footer

Include:

- About
- Shop
- Contact
- WhatsApp
- Payment information
- Privacy Policy
- Terms & Conditions
- Social links

PRODUCT LISTING PAGE

Create a Shop page.

Features:

- Product grid
- Search
- Category filtering
- Price filtering
- Size filtering
- Sorting
- Responsive design
- Pagination if needed

Each product card should show:

- Product image
- Product name
- Price
- Discount/original price if applicable
- Available sizes
- Stock status
- Quick View
- Add to Cart

PRODUCT DETAILS PAGE

This is very important.

Create a detailed product page.

Show:

- Large main product image
- Multiple product images
- Product name
- Price
- Description
- Available sizes
- Size selection
- Quantity selector
- Stock availability
- Add to Cart
- Buy Now
- Order through WhatsApp

SIZE FUNCTIONALITY

Each bangle product must support multiple sizes.

Support both adult and baby/children's sizes.

Example adult sizes:

- 2.2
- 2.4
- 2.6
- 2.8

Example baby/children's size categories:

- Newborn
- 0-6 Months
- 6-12 Months
- 1-2 Years
- 2-4 Years
- 4-6 Years
- 6-8 Years
- 8-10 Years

Do not assume these age ranges correspond to exact physical bangle measurements. The admin should be able to configure the actual age ranges.

Allow products to be categorized as:

- Adult
- Baby/Kids
- Unisex/All Ages where appropriate

The admin should be able to add custom size labels and measurements.

The customer MUST select a size before adding the product to the cart.

Do not allow adding a product without selecting an available size.

Store the selected size with the cart item and order.

CART

Create a shopping cart.

Cart should display:

- Product image
- Product name
- Selected size
- Price
- Quantity
- Subtotal
- Remove button

Calculate:

- Item subtotal
- Total quantity
- Grand total

Allow customers to:

- Increase quantity
- Decrease quantity
- Remove items
- Continue shopping
- Proceed to checkout

CHECKOUT

Create a clean checkout page.

Customer information:

- Full Name
- Phone Number
- Email
- Shipping Address
- City
- Postal Code
- Order Notes

Show an order summary.

PAYMENT METHODS

Provide payment options:

1. JazzCash
2. Bank Card

IMPORTANT:

Do not implement fake payment processing.

Structure the application so that a real payment gateway can be integrated securely.

For JazzCash:

- Create a proper payment-service abstraction/interface.
- Keep credentials/configuration in appsettings/user secrets/environment variables.
- Do not hardcode merchant credentials.
- Provide a clean integration point for JazzCash API/payment gateway.

For Bank Card:

- Create a payment service abstraction suitable for integrating a real card-payment provider such as Stripe or another supported provider.
- Never store raw card numbers, CVV, or sensitive card information in the database.
- Use the payment provider's hosted/secure checkout mechanism.

The checkout should allow:

Payment Method:

- () JazzCash
- () Bank Card

After payment, display:

- Payment status
- Order number
- Order summary

WHATSAPP INTEGRATION

Add WhatsApp ordering/contact functionality throughout the website.

Create a configurable WhatsApp business number in configuration.

Example:

WhatsAppNumber = "923XXXXXXX"

Do not hardcode the number throughout the application.

Add buttons such as:

"Order on WhatsApp"
"Chat on WhatsApp"

When clicked, open WhatsApp with a pre-filled message.

For example:

"Hello, I am interested in:
Product: [Product Name]
Size: [Selected Size]
Quantity: [Quantity]
Price: [Price]"

The WhatsApp link should work on both desktop and mobile.

DATABASE DESIGN

Use Entity Framework Core.

Create appropriate entities such as:

Product
ProductImage
ProductSize
Category
Cart
CartItem
Order
OrderItem
Customer
Payment
Address

Product should contain fields such as:

- Id
- Name
- Description
- Price
- DiscountPrice
- CategoryId
- IsActive
- CreatedAt

ProductSize should contain:

- Id
- ProductId
- Size
- SizeCategory (Adult/Baby/Kids/etc.)
- Measurement (optional)
- StockQuantity

OrderItem should store:

- ProductId
- ProductName snapshot
- SelectedSize
- Quantity
- UnitPrice
- TotalPrice

Do not rely only on the current product price when displaying historical orders.

ADMIN AREA

Create an admin area where an administrator can:

- Add products
- Edit products
- Delete/deactivate products
- Upload product images
- Add/remove available sizes
- Add adult/baby/kids size categories
- Configure size labels and measurements
- Set stock quantities
- Set prices
- Set discounts
- Manage categories
- View orders
- View order details
- Update order status

Order statuses can include:

- Pending
- Payment Pending
- Paid
- Processing
- Shipped
- Delivered
- Cancelled

SECURITY

Follow ASP.NET Core security best practices.

Use:

- Authentication
- Authorization
- Anti-forgery protection

- Model validation
- Server-side validation
- Parameterized database access through EF Core
- Secure configuration
- Proper error handling

Do not expose sensitive configuration values.

MVC STRUCTURE

Use a clean MVC structure such as:

Controllers/

HomeController.cs
ProductController.cs
CartController.cs
CheckoutController.cs
OrderController.cs
WhatsAppController.cs

Areas/

Admin/
 Controllers/
 Views/

Models/

Product.cs
ProductSize.cs
ProductImage.cs
Category.cs
Cart.cs
CartItem.cs
Order.cs
OrderItem.cs
Payment.cs
Customer.cs

ViewModels/

ProductDetailsViewModel.cs
CartViewModel.cs
CheckoutViewModel.cs
OrderViewModel.cs

Data/

ApplicationDbContext.cs

Services/

ProductService.cs
CartService.cs
OrderService.cs
Payment/
 IPaymentService.cs
 JazzCashPaymentService.cs
 CardPaymentService.cs
WhatsAppService.cs

Views/

Home/
Product/
Cart/
Checkout/
Order/
Shared/

wwwroot/

css/
js/
images/

UI INTERACTIONS

Add polished interactions such as:

- Product card hover effects
- Image zoom on product details
- Smooth image gallery switching
- Add-to-cart animation
- Cart count update
- Toast notifications
- Smooth scrolling
- Button loading states
- Mobile-friendly navigation

- Sticky cart/checkout summary where appropriate
- Smooth modal Quick View
- Size selection visual feedback

RESPONSIVENESS

The website must work properly on:

- Desktop
- Laptop
- Tablet
- Mobile

Pay special attention to:

- Product grids
- Navigation
- Product details
- Cart
- Checkout
- Payment selection
- WhatsApp buttons
- Baby/kids size selection on mobile

SEO

Add:

- Proper page titles
- Meta descriptions
- Semantic HTML
- SEO-friendly URLs
- Alt text for product images

ERROR HANDLING

Implement:

- Global exception handling
- User-friendly error pages
- Validation messages
- Proper logging

Do not expose stack traces to customers.

SEED DATA

Create seed data for demonstration.

Include at least:

- 10 sample bangle products
- Multiple categories
- Adult and baby/kids products
- Multiple sizes per product
- Product descriptions
- Prices
- Sample product images/placeholders

Make the application runnable immediately after database migration.

CONFIGURATION

Use appsettings.json for non-sensitive configuration.

Example structure:

```
"StoreSettings": {
  "StoreName": "Your Bangle Store",
  "WhatsAppNumber": "923XXXXXXXXX",
  "Currency": "PKR"
}
```

Payment credentials must NOT be hardcoded.

Use:

- User Secrets for development
- Environment variables / secure secret storage for production

IMPORTANT IMPLEMENTATION RULES

1. First inspect the existing project structure before creating files.
2. Do not unnecessarily overwrite existing working code.
3. Follow existing naming conventions where possible.
4. Use dependency injection.
5. Use async/await for database operations.

6. Use ViewModels instead of exposing database entities directly to views where appropriate.
7. Use EF Core migrations.
8. Keep controllers thin and move business logic into services.
9. Validate all user input.
10. Do not store raw card information.
11. Do not put payment secrets in source code.
12. Do not create fake payment success responses.
13. Make WhatsApp functionality configurable.
14. Keep the design polished and production-like.
15. Make the site functional, not just a static UI.
16. Make adult and baby/kids sizing flexible and configurable rather than hardcoding measurements.

FINAL DELIVERABLE

Build the complete working application.

After implementation:

1. Ensure the project builds successfully.
2. Fix compilation errors.
3. Create/update EF Core migrations.
4. Ensure the database can be created.
5. Test navigation between pages.
6. Test adult and baby/kids product size selection.
7. Test cart functionality.
8. Test checkout validation.
9. Test WhatsApp links.
10. Verify responsive behavior.
11. Explain how to configure:
 - SQL Server
 - WhatsApp number
 - JazzCash credentials
 - Card payment provider
12. Provide the commands required to run the project.

Do not stop at creating a visual mockup. Implement the actual MVC functionality and make the application runnable.